Coding: React JS TypeScript Standards

ReactJsTypeScriptStandards

```

---
applyTo: "*.tsx,*.ts"
---

# React Best Practices Knowledge Base

This knowledge base serves as a comprehensive guide when working on React projects that the AI should follow. It outlines industry-standard best practices for React development, covering component structure, hooks, state management, code quality, performance optimization, and more.

## 1. Component Structure and Organization

### 1.1 Break Down the UI into Components
- Divide your UI into small, reusable components that each serve a single purpose
- Follow the Single Responsibility Principle: each component should do one thing well
- Create a component hierarchy that mirrors your visual hierarchy
- Extract repeated UI patterns into reusable components

### 1.2 Use Meaningful Component Names
- Use clear, descriptive names that reflect the component's purpose
- Follow PascalCase naming convention for components (e.g., `UserProfile`, `NavigationBar`)
- Be specific but concise (e.g., `ProductCard` instead of just `Card`)
- Avoid abbreviations unless they're universally understood

### 1.3 Organize Components Logically
- Group related components in dedicated folders
- Consider feature-based organization for larger applications
- Common organizational patterns:
  - By feature (e.g., `/auth`, `/dashboard`, `/products`)
  - By component type (e.g., `/pages`, `/components`, `/hooks`)
- Keep component files small and focused
- Separate container (smart) components from presentational (dumb) components
- Place interfaces and types in a dedicated `interfaces/` folder.

### 1.4 Implement Component Composition
- Favor composition over inheritance
- Use children props to create flexible, reusable components
- Create compound components for complex UIs
- Use higher-order components (HOCs) and render props when appropriate

## 2. Functional Components and Hooks

### 2.1 Prefer Functional Components
- Use functional components with hooks instead of class components
- Functional components are more concise and easier to test
- They encourage better patterns and code organization
- They simplify handling of props and state

### 2.2 Follow Hook Rules
- Only call hooks at the top level of your component
- Don't call hooks inside loops, conditions, or nested functions
- Only call hooks from React function components or custom hooks
- Use the React ESLint plugin to enforce these rules
- Understand the dependency array in `useEffect` and other hooks

### 2.3 Create Custom Hooks for Logic Reuse
- Extract repetitive logic into custom hooks
- Name custom hooks with the `use` prefix (e.g., `useFetch`, `useForm`)
- Keep custom hooks focused on a specific concern
- Design hooks to be composable and reusable across components
- Examples of common custom hooks:
  - `useLocalStorage` for persistent state
  - `useForm` for form handling
  - `useFetch` for data fetching
  - `useMediaQuery` for responsive design

### 2.4 Use Built-in Hooks Effectively
- `useState` for local component state
- `useEffect` for side effects (API calls, subscriptions)
- `useContext` for accessing context
- `useReducer` for complex state logic
- `useCallback` and `useMemo` for performance optimization
- `useRef` for persistent values and DOM references
```

3. State Management

3.1 Keep State Minimal
- Store only the essential data in state
- Derive calculated values instead of storing them
- Keep related state together
- Avoid duplicating state from props
- Use the principle of "single source of truth"

3.2 Centralize State When Necessary
- Use React Context for state that needs to be accessed by many components
- Consider state management libraries (Zustand, Jotai) for complex applications
- Implement proper state initialization
- Follow these guidelines for choosing state management:
  - Local component state for UI state
  - Context for shared state in a subtree
  - Global state management for application-wide state

3.3 Implement Proper Data Flow
- Follow React's unidirectional data flow
- Pass data down through props
- Use callbacks for communication from child to parent
- Avoid prop drilling by using Context or state management libraries
- Keep state as close as possible to where it's used

4. Code Quality

4.1 Write DRY Code
- Don't Repeat Yourself: extract repeated logic into reusable functions or components
- Create utility functions for common operations
- Use higher-order components or custom hooks for cross-cutting concerns
- Identify patterns in your code and abstract them

4.2 Use Destructuring
- Destructure props and state for cleaner code
- Use object destructuring in function parameters
- Destructure arrays when appropriate
- Provide default values when destructuring

```jsx
// Instead of
function UserProfile(props) {
  return <h1>{props.user.name}</h1>;
}

// Do this
function UserProfile({ user: { name } }) {
  return <h1>{name}</h1>;
}
```

4.3 Maintain a Proper Import Structure
- Group imports by type (React, third-party, local)
- Order imports consistently
- Use absolute imports for better readability
- Consider using barrel files (index.js) to simplify imports

```jsx
// React imports
import React, { useState, useEffect } from 'react';

// Third-party imports
import { useQuery } from 'react-query';
import classNames from 'classnames';

// Local imports
import Button from 'components/Button';
import { formatDate } from 'utils/dateUtils';
```

4.4 Use TypeScript for Type Safety
- Use TypeScript for all new code
- Follow functional programming principles where possible
- Define interfaces for component props
- Use type annotations for state and variables
- Leverage TypeScript's utility types (Partial, Omit, Pick, etc.)
- Create reusable type definitions
- Use strict type checking
- Avoid using `any`; use `unknown` or proper types.
- Use enums for fixed sets of values.

```tsx
```

```
interface UserProps {
  id: string;
  name: string;
  email: string;
  role?: 'admin' | 'user';
}

function User({ id, name, email, role = 'user' }: UserProps) {
  // Component implementation
}
```

4.5 Use Prettier and ESLint
- Configure Prettier for consistent code formatting
- Set up ESLint with React-specific rules
- Use the React Hooks ESLint plugin
- Integrate with your IDE for real-time feedback
- Run linting as part of your CI/CD pipeline

4.6 Write Meaningful Comments
- Focus on explaining "why" rather than "what"
- Document complex logic and business rules
- Use JSDoc for component props and functions
- Keep comments up-to-date with code changes
- Consider using TypeScript instead of comments for type documentation

5. Performance Optimization

5.1 Avoid Side Effects in Render
- Keep render functions pure
- Move side effects to appropriate lifecycle methods or hooks
- Use `useEffect` for data fetching, subscriptions, and DOM manipulations
- Handle asynchronous operations properly

5.2 Use Keys for Lists
- Always provide a unique `key` prop when rendering lists
- Use stable IDs as keys, not array indices
- Keys help React identify which items have changed, been added, or removed
- Proper keys improve reconciliation performance

```jsx
{users.map(user => (
  <UserItem key={user.id} user={user} />
))}
```

5.3 Use React.memo and useMemo
- Wrap components with `React.memo()` to prevent unnecessary re-renders
- Use `useMemo` to memoize expensive calculations
- Use `useCallback` to memoize event handlers
- Be strategic about memoization—don't overuse it
- Understand the dependency arrays in `useMemo` and `useCallback`

```jsx
// Memoize expensive calculations
const sortedItems = useMemo(() => {
  return [...items].sort((a, b) => a.value - b.value);
}, [items]);

// Memoize event handlers
const handleClick = useCallback(() => {
  console.log('Clicked!', id);
}, [id]);
```

5.4 Implement Code Splitting
- Use dynamic imports with `React.lazy()` for code splitting
- Implement route-based code splitting
- Use Suspense for loading states
- Consider component-level code splitting for large components

```jsx
const LazyComponent = React.lazy(() => import('./LazyComponent'));

function MyComponent() {
  return (
    <React.Suspense fallback={<Loading />}>
      <LazyComponent />
    </React.Suspense>
  );
}
```

5.5 Optimize Rendering
- Avoid unnecessary re-renders
- Use the React DevTools Profiler to identify performance issues
- Implement virtualization for long lists (react-window, react-virtualized)
- Consider using Web Workers for CPU-intensive tasks
- Use throttling and debouncing for frequent events

6. Other Recommendations

6.1 Think in Interfaces
- Design components based on their public API
- Define clear contracts between components
- Use TypeScript interfaces to document component props
- Consider how components will be used by other developers

6.2 Minimize Duplicate JSX and State
- Extract repeated JSX into separate components
- Use composition to avoid duplication
- Lift state up when multiple components need the same data
- Use context or state management for shared state

6.3 Prefer Handlers for Side Effects
- Handle side effects in event handlers when possible
- Use `useEffect` for side effects that aren't tied to specific events
- Keep side effects isolated and testable
- Handle errors properly in async operations

6.4 Keep React and Dependencies Up-to-Date
- Regularly update React and its ecosystem
- Follow a strategy for dependency updates
- Use tools like Dependabot or Renovate
- Stay informed about breaking changes
- Test thoroughly after updates

6.5 Implement Error Boundaries
- Use error boundaries to catch JavaScript errors
- Provide fallback UI for error states
- Log errors for monitoring
- Implement graceful degradation

```jsx
class ErrorBoundary extends React.Component {
  state = { hasError: false };

  static getDerivedStateFromError(error) {
    return { hasError: true };
  }

  componentDidCatch(error, errorInfo) {
    logErrorToService(error, errorInfo);
  }

  render() {
    if (this.state.hasError) {
      return <h1>Something went wrong.</h1>;
    }

    return this.props.children;
  }
}
```

6.6 Implement Accessibility
- Use semantic HTML elements
- Provide proper ARIA attributes
- Ensure keyboard navigation works
- Test with screen readers
- Follow WCAG guidelines

6.7 Write Tests
- Implement unit tests for components and hooks
- Use React Testing Library for component testing
- Test user interactions and edge cases
- Implement integration tests for complex features
- Aim for good test coverage

References

This knowledge base incorporates best practices from the following sources:

1. [React Best Practices to Improve Your Code](https://medium.com/@onix_react/react-best-practices-to-improve-your-code-a4c68962d5dd)
2. [20 ReactJS Best Practices Learned from Code Reviews](https://medium.com/@rana.adnanali/20-reactjs-best-practices-learned-from-code-reviews-9f846a132e52)

```
3. [Best Practices and Design Patterns in React JS for High-Quality Applications](https://medium.com/@obrm770/best-practices-and-design-patterns-in-react-js-for-high-quality-applications-6b203be747fb)
```